

LQR as a Quadratic Program

KKT Structure and Riccati Recursion

RCI Lab

Kyung Hee University

February 21, 2026

Finite-Horizon LQR Problem

$$\min_{\{u_k\}} \sum_{k=0}^{N-1} x_k^T Q x_k + u_k^T R u_k + x_N^T Q_f x_N$$

$$x_{k+1} = A x_k + B u_k$$

$$Q \succeq 0, \quad R \succ 0$$

Initial state x_0 given.

Example: Double Integrator

State:

$$x = \begin{bmatrix} \textit{position} \\ \textit{velocity} \end{bmatrix}$$

Discrete dynamics:

$$A = \begin{bmatrix} 1 & h \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ h \end{bmatrix}$$

Brick sliding on ice (no friction).

LQR as a QP

Stack decision variables:

$$z = \begin{bmatrix} x_1 \\ \vdots \\ x_N \\ u_0 \\ \vdots \\ u_{N-1} \end{bmatrix}$$

Cost can be written as:

$$\frac{1}{2} z^T H z$$

Dynamics constraints:

$$Cz = d$$

Thus LQR is standard QP:

KKT Conditions for LQR QP

Lagrangian:

$$\mathcal{L}(z, \lambda) = \frac{1}{2} z^T H z + \lambda^T (Cz - d)$$

KKT system:

$$Hz + C^T \lambda = 0$$

$$Cz = d$$

One large linear system.

Structure of the KKT Matrix

KKT matrix:

$$\begin{bmatrix} H & C^T \\ C & 0 \end{bmatrix}$$

Properties:

- Sparse - Block banded - Quasi-definite

Direct solve complexity:

$$\mathcal{O}((N(n+m))^3)$$

Too expensive for large horizon.

Exploiting Structure → Riccati

Instead of solving one big system:

Use dynamic programming structure.

Assume value function quadratic:

$$V_k(x) = x^T P_k x$$

Backward recursion yields:

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B K_k$$

This is the Riccati recursion.

Control law:

$$u_k = -K_k x_k$$

Key insight:

Riccati recursion = Structured elimination of KKT system.

Much cheaper:

$$\mathcal{O}(N(n+m)^3)$$

instead of cubic in horizon length.

Backward + Forward Pass

Algorithm:

Backward pass: Compute P_k, K_k

Forward rollout: Compute state and control

$$x_{k+1} = (A - BK_k)x_k$$

Exact solution of QP.

Why Better Than Shooting?

Indirect shooting:

- Integrates costate - Sensitive to initial guess - No feedback during rollout

Riccati:

- Produces feedback gain - Stabilizes system - Rejects disturbances

Huge advantage in practice.

Infinite-Horizon LQR

If system time-invariant:

$$A_k = A, \quad B_k = B$$

As $N \rightarrow \infty$:

$$P_k \rightarrow P$$

Solve Algebraic Riccati Equation (DARE):

$$P = Q + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A$$

Steady-state gain:

$$K = (R + B^T P B)^{-1} B^T P A$$

Used for:

- Stabilization - Regulation - MPC terminal cost

Most control libraries implement:

`dare()`

LQR:

- Convex optimal control - Equivalent to QP - KKT system has structure - Riccati = structured solver

Foundation for:

- iLQR - DDP - MPC - Whole-body control